CSE221: Algorithms

Minimum Spanning Tree

Prepared by: A B M Muntasir Rahman [MUNR]



Spanning Tree

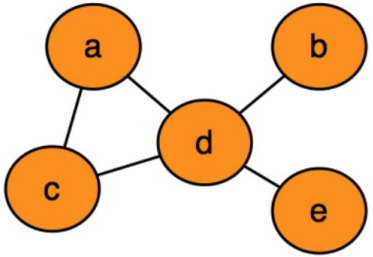
- A spanning tree is a subgraph/tree of a **connected, undirected graph** that:
  - Includes all the vertices of the original graph.
  - Is a tree (i.e. no disjoint vertices and has no cycles).
  - Uses only a subset of edges.

- If the graph has N vertices, then any spanning tree of the graph will exactly have **N - 1 edges.**

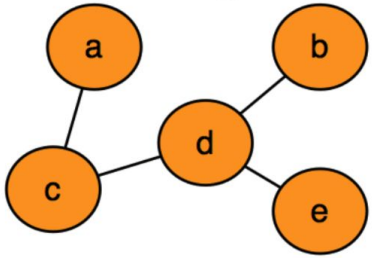
- **A graph can have many different spanning trees.**

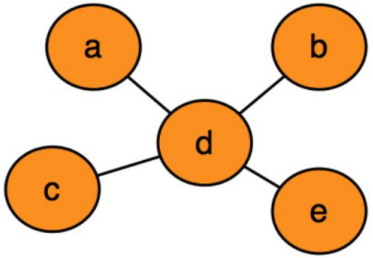
Spanning Tree

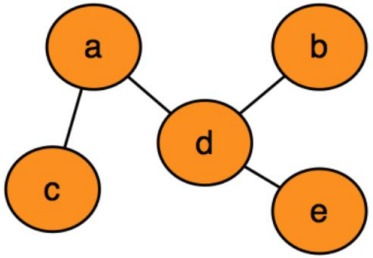
Given the following graph:



The spanning trees are:







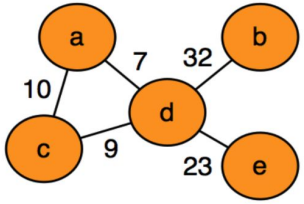
Minimum Spanning Tree (MST)

- If the graph is **weighted** (i.e. each edge has a cost/weight), then a minimum spanning tree (MST) is:
  - A spanning tree with **minimum total edge weight** among all possible spanning trees.

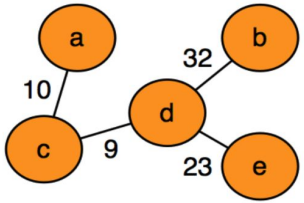
- **An MST may not be unique if multiple spanning trees have the same total weight.**

Minimum Spanning Tree (MST)

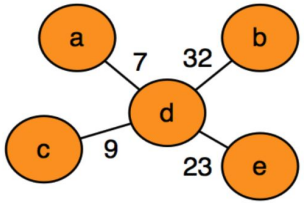
Given the following graph:



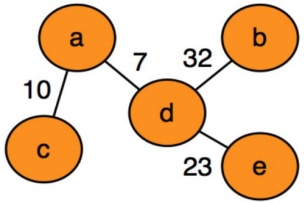
The spanning trees (with associated total weights) are:



$\sum_{e \in T} w(e) = 74$



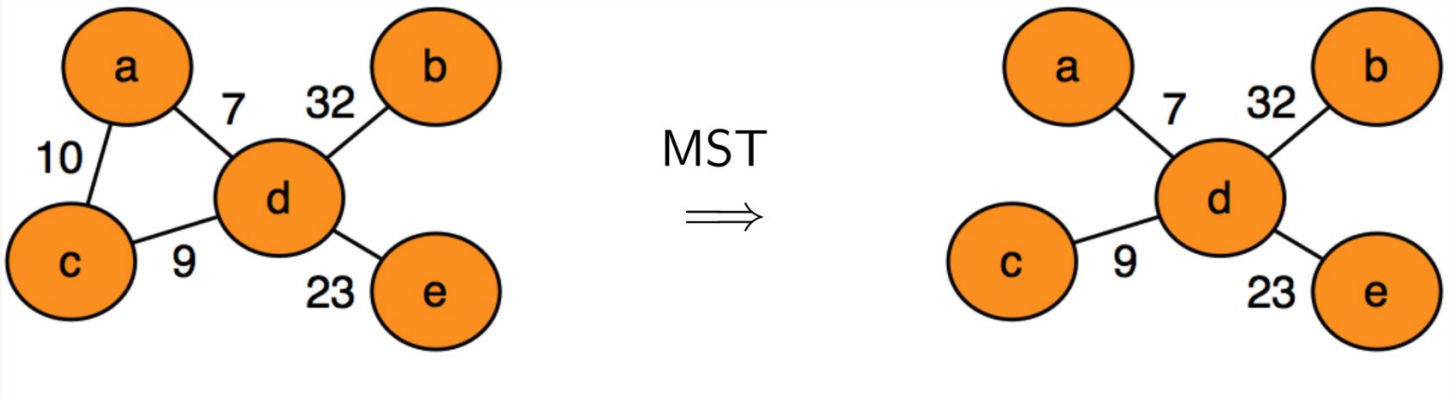
$\sum_{e \in T} w(e) = 71$



$\sum_{e \in T} w(e) = 72$

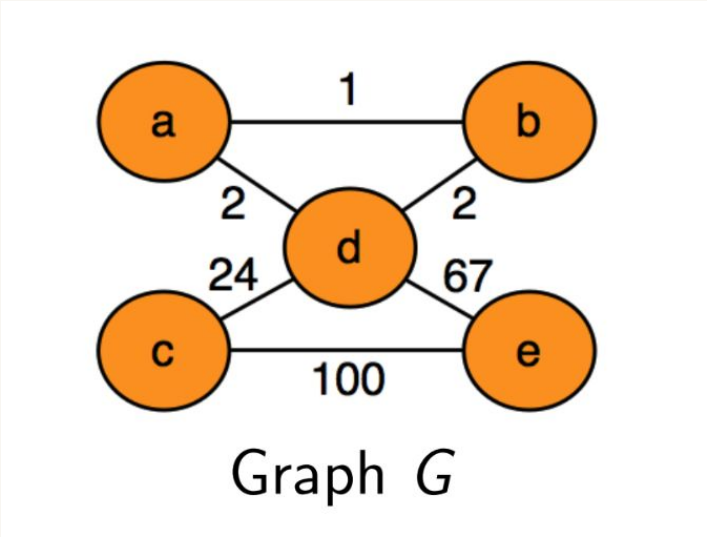
Minimum Spanning Tree (MST)

- Of the spanning trees we got in the previous slide, the **second spanning tree** has the least overall weight - and it is the **Minimum Spanning Tree!**

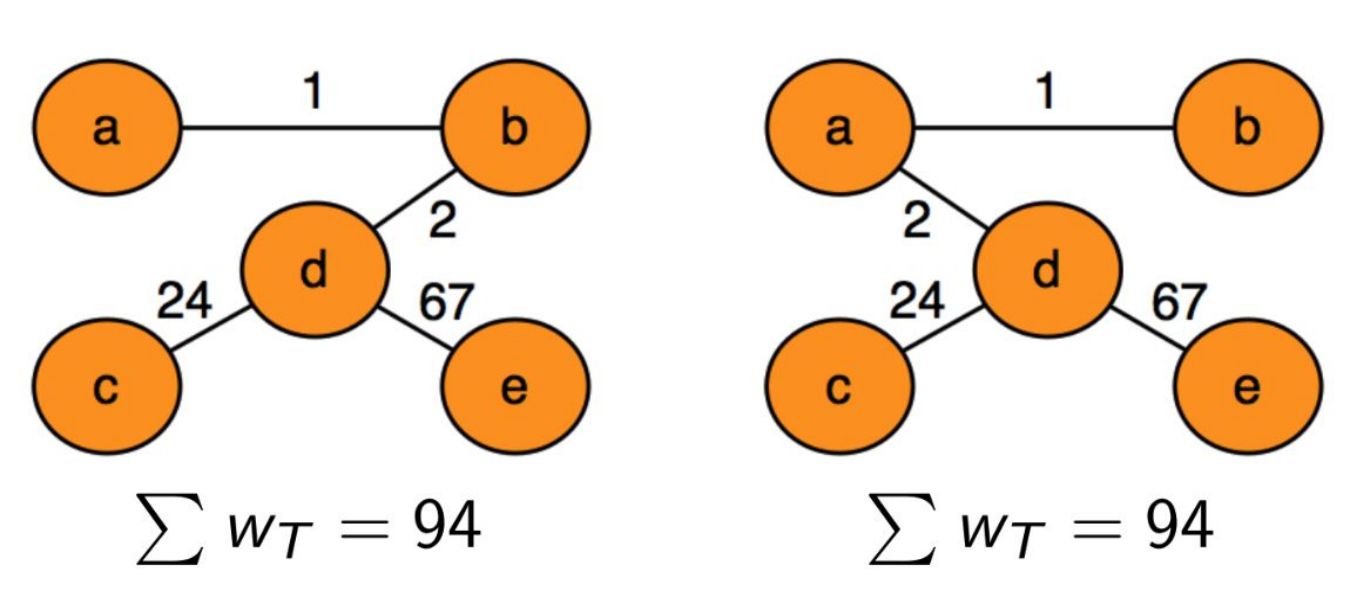


Uniqueness of MST

- **The minimum cost spanning tree may not be unique!**



Uniqueness of MST



However, if the weights are all distinct (i.e. $w(u_i, v_j) \neq w(u_k, v_l)$), then it is indeed unique.

Applications of MST

- **Network Design** (Telecommunication networks/Electrical grids etc.)
- **Transportation & Road Networks** (Planning airline routines/delivery networks/trip itinerary etc.)
- **Circuit Design** (Minimizing total length of wiring) etc.

In short: MST is applied anywhere we want to **connect things efficiently while minimizing cost and avoiding cycles**.

How to find MST?

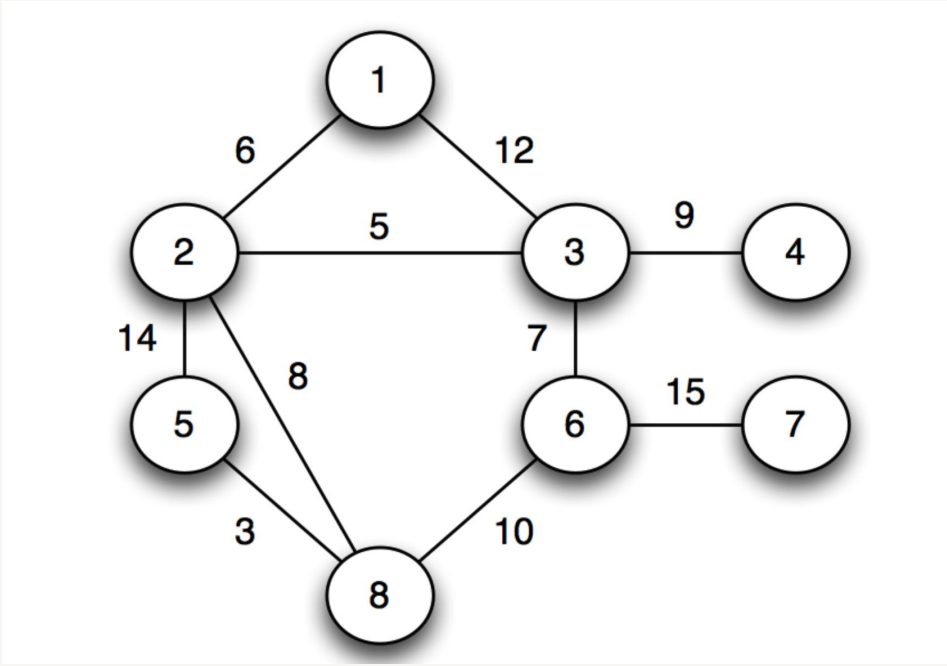
- There are two algorithms: **Prim's Algorithm and Kruskal's Algorithm** which can find MST in a **weighted and undirected** graph.

- In this course, we will discuss about Kruskal's Algorithm only.

Kruskal's Algorithm

```
MST-KRUSKAL(G, w)
1  A ← ∅
2  for each vertex v ∈ V[G]
3        do MAKE-SET(v)
4  sort the edges of E into non-decreasing order by weight w
5  for each edge (u, v) ∈ E, taken in non-decreasing order by weight
6        do if FIND-SET(u) ≠ FIND-SET(v)
7              then A ← A ∪ {(u, v)}
8                   UNION(u, v)
9  return A
```

Kruskal's Algorithm (Simulation)



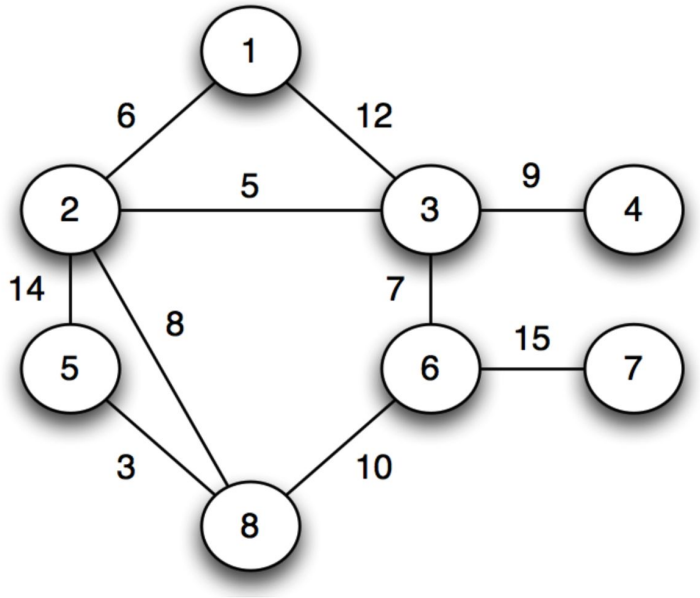
Kruskal's Algorithm (Simulation)

$w(u, v)$	(u, v)
3	(5,8)
5	(2,3)
6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)

$|V| = 8$

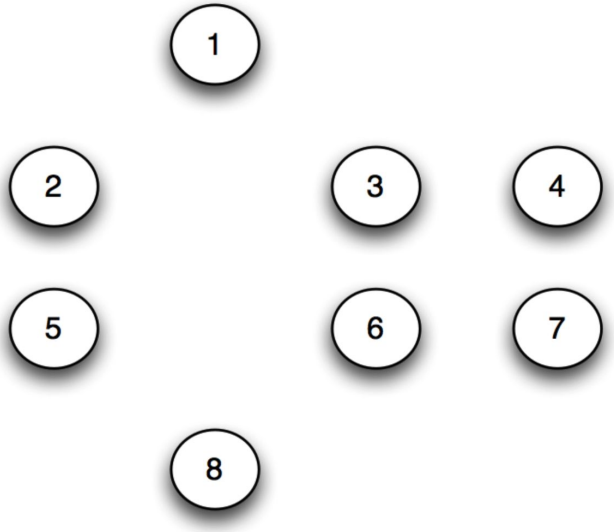
$|E| = 10$

$|T| = 0$



Kruskal's Algorithm (Simulation)

$w(u,v)$	(u,v)
3	(5,8)
5	(2,3)
6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

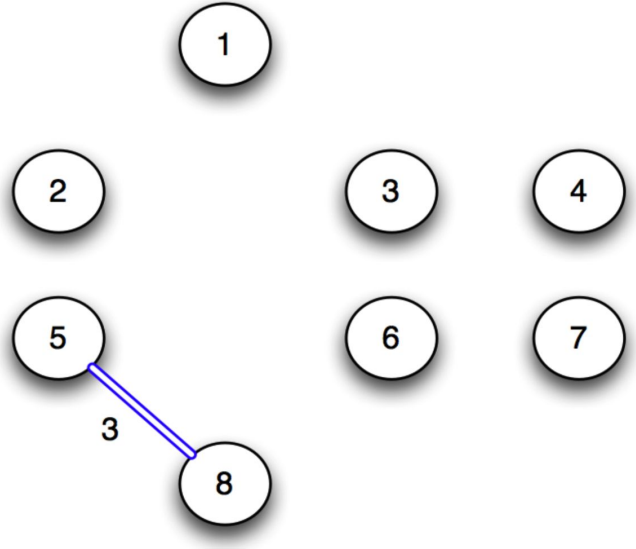
$|T| = 0$

Vertex sets:

{1},{2},{3},{4},{5},{6},{7},{8}

Kruskal's Algorithm (Simulation)

$w(u, v)$	(u, v)
3	(5,8)
5	(2,3)
6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

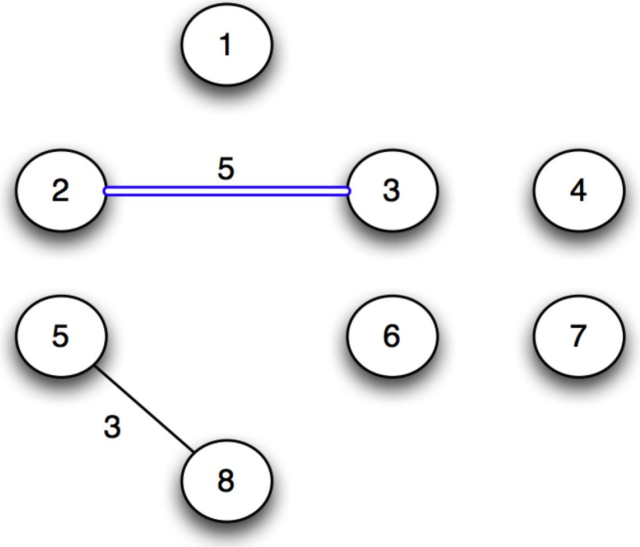
$|T| =$ 1

Vertex sets:

$\{1\},\{2\},\{3\},\{4\},\{5\},\{6\},\{7\},\{8\} \Longrightarrow \{1\},\{2\},\{3\},\{4\},$ {5, 8} $,\{6\},\{7\}$

Kruskal's Algorithm (Simulation)

$w(u,v)$	(u,v)
✓ 3	(5,8)
5	(2,3)
6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

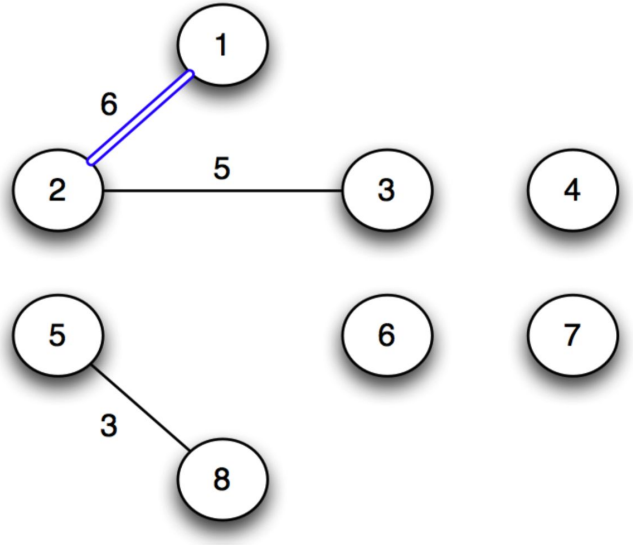
$|T| = 2$

Vertex sets:

$\{1\},\{2\},\{3\},\{4\},\{5, 8\},\{6\},\{7\} \Longrightarrow \{1\},\{2, 3\},\{4\},\{5, 8\},\{6\},\{7\}$

Kruskal's Algorithm (Simulation)

$w(u,v)$	(u,v)
✓ 3	(5,8)
✓ 5	(2,3)
6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

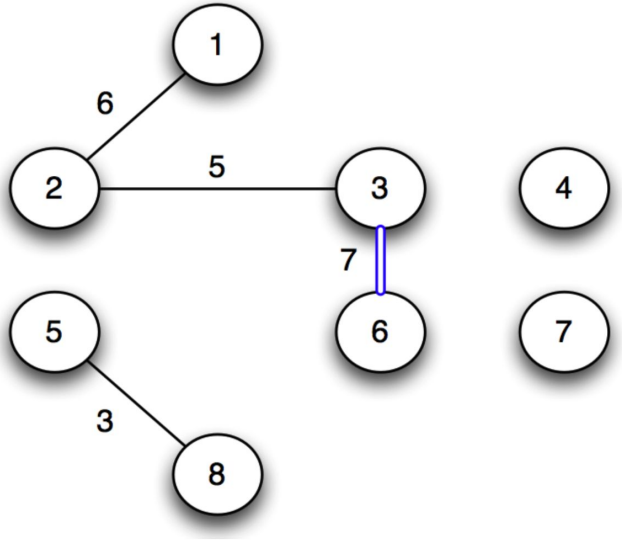
$|T| =$ 3

Vertex sets:

$\{1\},\{2, 3\},\{4\},\{5, 8\},\{6\},\{7\} \Longrightarrow$ $\{1, 2, 3\}$ $,\{4\},\{5, 8\},\{6\},\{7\}$

Kruskal's Algorithm (Simulation)

$w(u,v)$	(u,v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

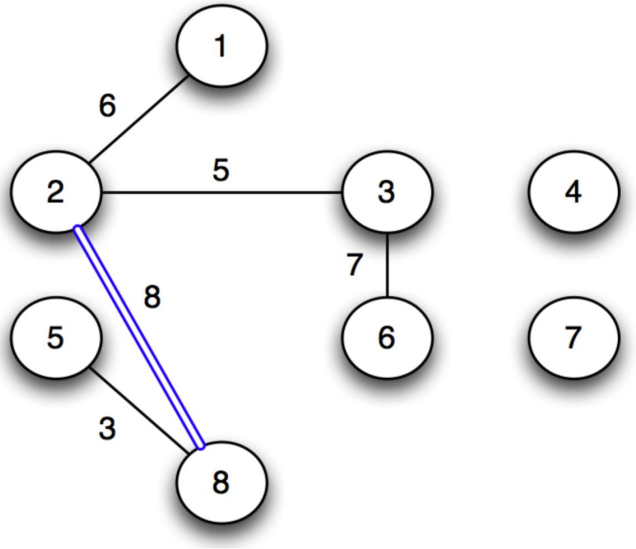
$|T| =$ 4

Vertex sets:

$\{1, 2, 3\},\{4\},\{5, 8\},\{6\},\{7\} \Longrightarrow$ $\{1, 2, 3, 6\}$ $,\{4\},\{5, 8\},\{7\}$

Kruskal's Algorithm (Simulation)

$w(u,v)$	(u,v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

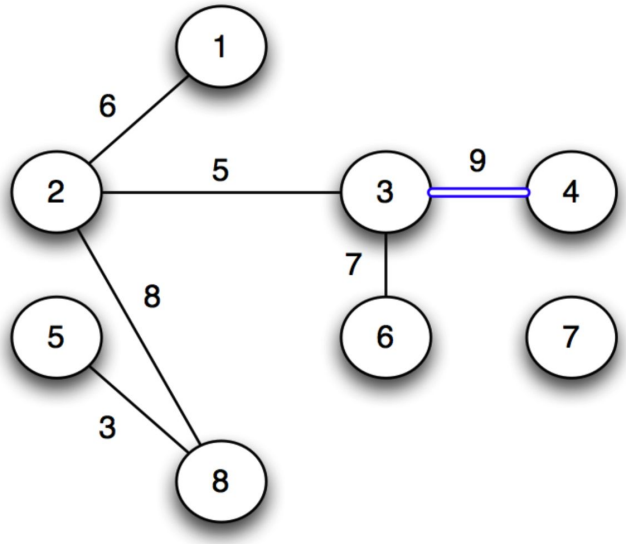
$|T| = \boxed{5}$

Vertex sets:

$\{1, 2, 3, 6\},\{4\},\{5, 8\},\{7\} \Longrightarrow \boxed{\{1, 2, 3, 5, 6, 8\}},\{4\},\{7\}$

Kruskal's Algorithm (Simulation)

$w(u,v)$	(u,v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

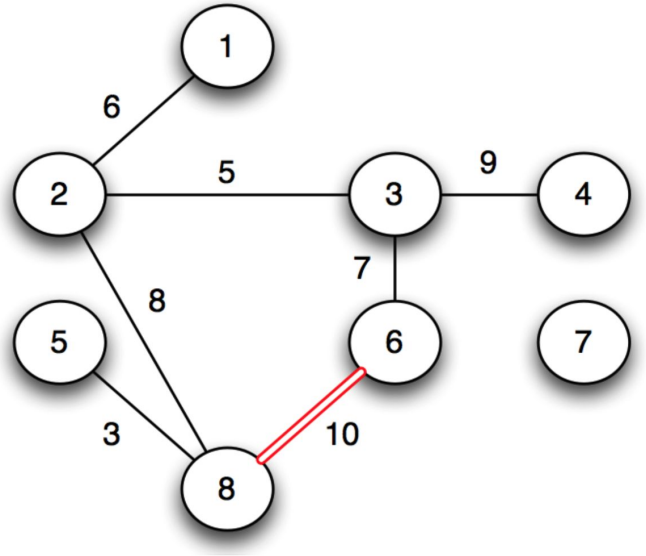
$|T| =$ 6

Vertex sets:

$\{1, 2, 3, 5, 6, 8\}, \{4\}, \{7\} \Longrightarrow$ $\{1, 2, 3, 4, 5, 6, 8\}, \{7\}$

Kruskal's Algorithm (Simulation)

$w(u,v)$	(u,v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

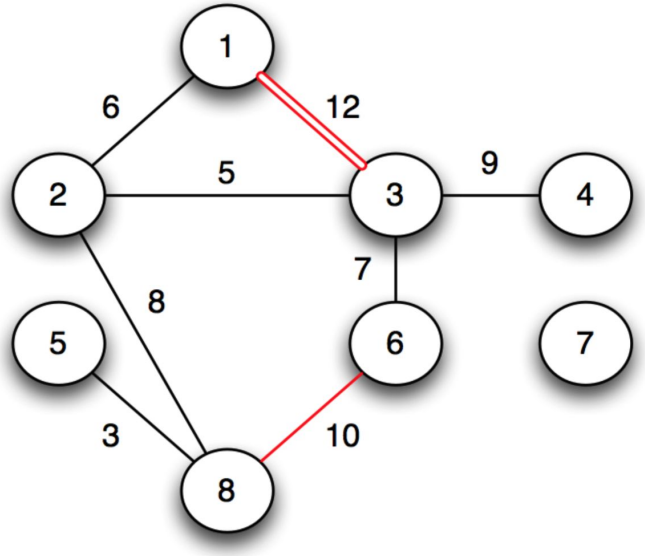
$|T| = \boxed{6}$

Vertex sets:

$\{1, 2, 3, 4, 5, 6, 8\}, \{7\} \Longrightarrow \{1, 2, 3, 4, 5, 6, 8\}, \{7\}$

Kruskal's Algorithm (Simulation)

$w(u,v)$	(u,v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
× 10	(6,8)
12	(1,3)
14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

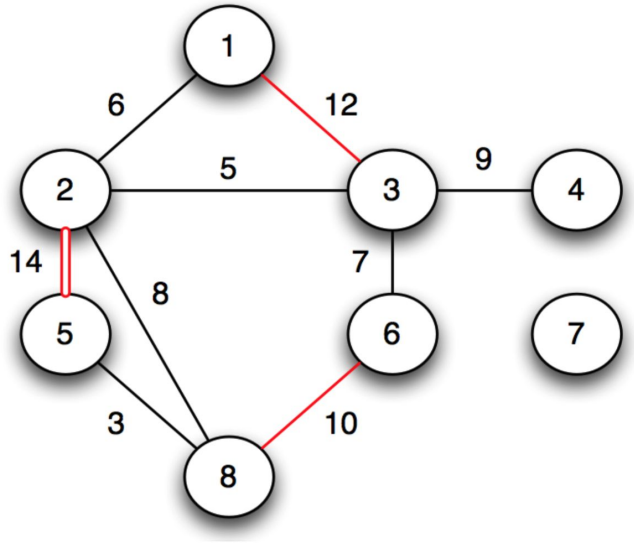
$|T| = \boxed{6}$

Vertex sets:

$\{1, 2, 3, 4, 5, 6, 8\}, \{7\} \Longrightarrow \{1, 2, 3, 4, 5, 6, 8\}, \{7\}$

Kruskal's Algorithm (Simulation)

$w(u,v)$	(u,v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
× 10	(6,8)
× 12	(1,3)
14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

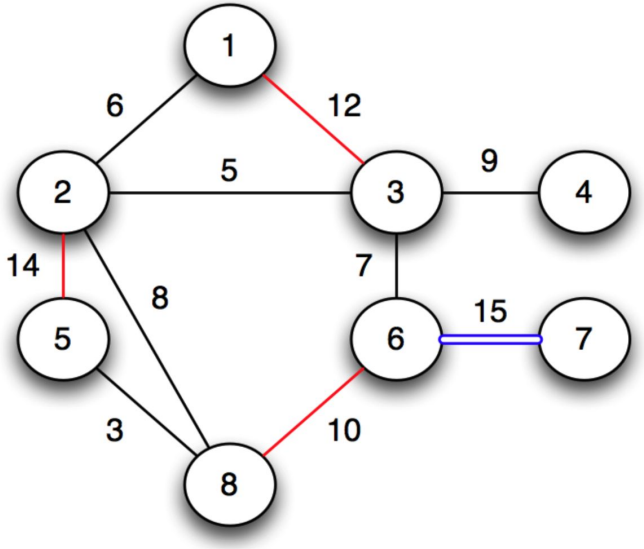
$|T| = 6$

Vertex sets:

$\{1, 2, 3, 4, 5, 6, 8\},\{7\} \Longrightarrow \{1, 2, 3, 4, 5, 6, 8\},\{7\}$

Kruskal's Algorithm (Simulation)

$w(u,v)$	(u,v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
× 10	(6,8)
× 12	(1,3)
× 14	(2,5)
15	(6,7)



$|V| = 8$

$|E| = 10$

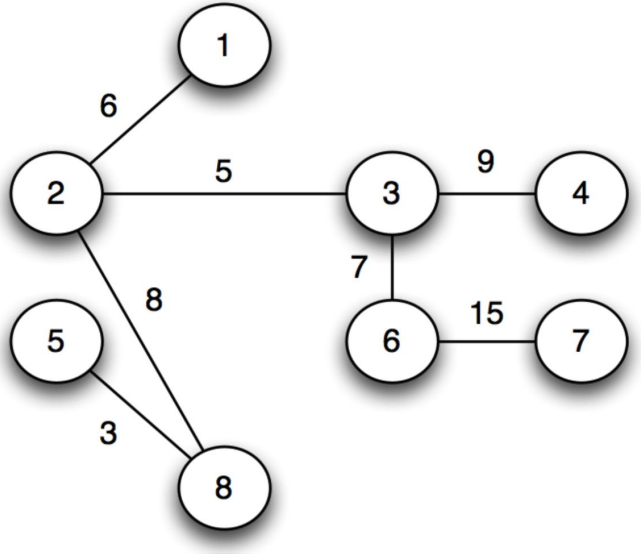
$|T| =$ 7

Vertex sets:

$\{1, 2, 3, 4, 5, 6, 8\}, \{7\} \Longrightarrow \{1, 2, 3, 4, 5, 6, 7, 8\}$

Kruskal's Algorithm (Simulation)

$w(u,v)$	(u,v)
✓ 3	(5,8)
✓ 5	(2,3)
✓ 6	(1,2)
✓ 7	(3,6)
✓ 8	(2,8)
✓ 9	(3,4)
× 10	(6,8)
× 12	(1,3)
× 14	(2,5)
✓ 15	(6,7)



$|V| = 8$

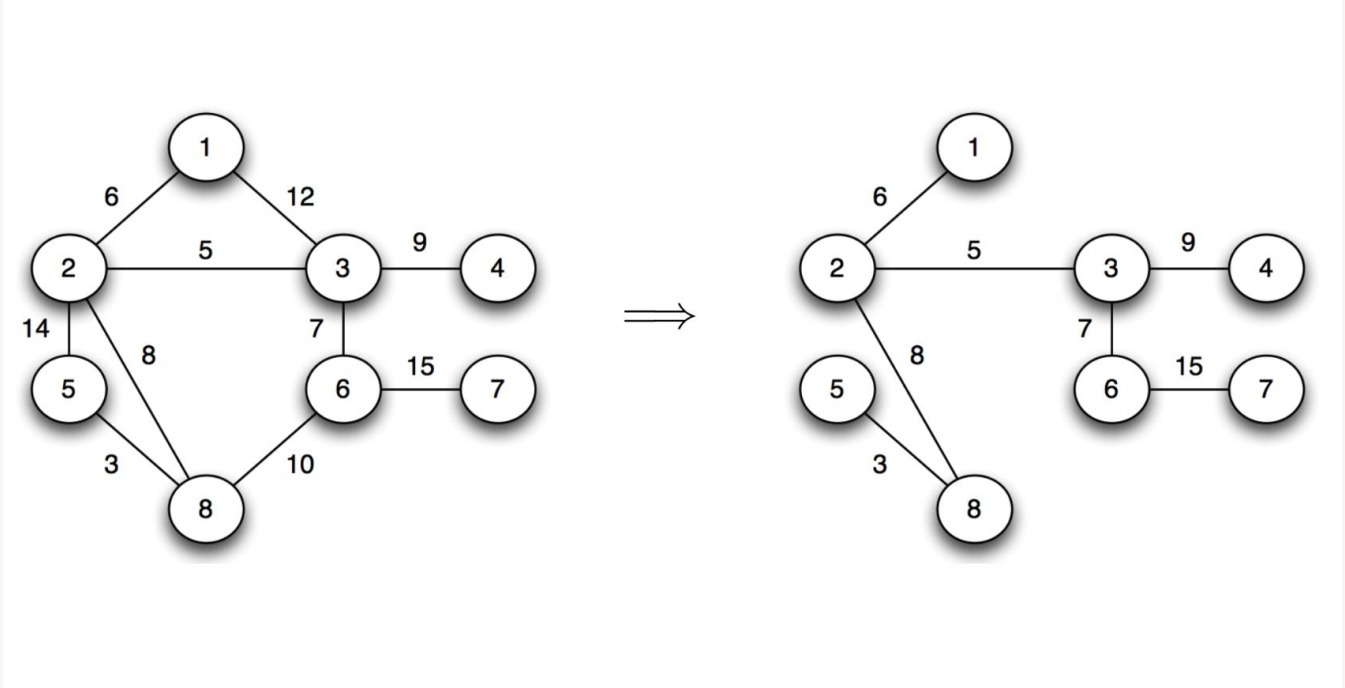
$|E| = 10$

$|T| = 7$

Vertex sets:

$\{1, 2, 3, 4, 5, 6, 7, 8\}$

Kruskal’s Algorithm (Simulation)



Pseudocode

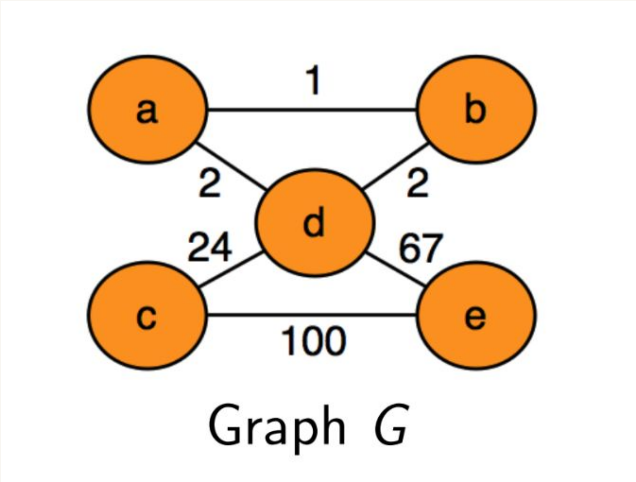
```
KRUSKAL(G):
    edges = all edges in G, sorted by weight (ascending)
    //Initialize DSU
    parent[v] = v, rank[v] = 0 for each vertex v
    mst    = []
    total  = 0
    for each edge (u, v, w) in edges:
        if FIND(u) ≠ FIND(v):
            UNION(u, v)
            mst.append((u, v, w))
            total += w
        if len(mst) == len(V) - 1:
            break
    return mst, total
```

```
FIND(x):
   if parent[x] ≠ x:
       parent[x] = FIND(parent[x])
   return parent[x]

UNION(x, y):
    rx = FIND(x)
    ry = FIND(y)
    if rx == ry: return
    if rank[rx] < rank[ry]:
        parent[rx] = ry
    else if rank[rx] > rank[ry]:
        parent[ry] = rx
    else:
        parent[ry] = rx, rank[rx] += 1
```

Approach

- Kruskal's Algorithm follows the **Greedy Approach** to find the MST in a **weighted and undirected graph.** At every step of the solution, **Kruskal's algorithm picks the edges with the with the least cost/weight unless it makes a cycle. (i.e. greedy choice)**



Graph *G*

For this graph, Kruskal will choose the edge (a, b) first because it has the least cost/weight. Then it will choose either edge (a, d) or edge (b, d) as both has same cost/weight. In this way, it will choose least cost/weighted edges unless it makes a cycle and will eventually build up the MST (i.e. greedy approach).

Properties

- **Time Complexity: O(E log E)**
  - Sorting edges takes **O(E log E)** (where E = number of edges).
  - Union-Find operations take almost constant time **(O(α(V))**, inverse Ackermann function, which is very small).
- **Space Complexity: O(E + V)**
  - Storing sorted edges takes **O(E)** space.
  - Union - Find operations takes **O(V)** space for rank/parent array.
- Follows **greedy approach**
- Uses **Disjoint Set Union (DSU)** or **Union-Find** data structure to efficiently check whether adding an edge forms a cycle.
- If the graph is **not connected**, it produces a **Minimum Spanning Forest** (MST for each component).
- **Does not work for directed graphs as path can not be guaranteed.**

Reference

- **Graph Algorithms by Mumit Khan**
  https://drive.google.com/file/d/1MCbK6w6tSITZ3UrV9Na4-tPTolKXjO3D/view?usp=sharing